SYSTEMVERILOG

1. What is the difference between blocking and non-blocking assignments?
2. How does fork-join work in Verilog?
3. Explain race conditions in Verilog and how to avoid them.
4. What is the difference between initial and always blocks?
5. How does a generate block work in Verilog?
6. What are virtual interfaces and why do we need them?
7. Explain the difference between rand and randc.
8. What is the significance of constraint_mode()?
9. How do you use covergroup in SystemVerilog?
10. What are the different types of assertions in SystemVerilog?
11. What is the UVM factory and why is it used?
12. Explain the uvm_component and uvm_object hierarchy.
13. What is the difference between uvm_analysis_port and uvm_tlm_analysis_fifo?
14. How does the uvm_config_db mechanism work?
15. What is the role of uvm_phase and how does uvm_run_phase differ from uvm_main_phase
16. What is the difference between a class and a struct?
17. Explain the concept of inheritance in SystemVerilog.
18. What are virtual methods and virtual classes?
19. How do you achieve polymorphism in SystemVerilog?
20. What is a typedef in SystemVerilog, and why is it useful?
21. What are the different types of assertions in SystemVerilog?
22. Explain immediate vs. concurrent assertions.
23. What is the difference between covergroup and coverpoint?
24. How do you write a SystemVerilog assertion for a handshake protocol?
25. How can you disable an assertion dynamically?
26. What is the purpose of an interface in SystemVerilog?
27. How do you use modport inside an interface?

28. What is the difference between virtual interface and interface?
29. How do interfaces help in testbench design?
30. How can you override parameters in SystemVerilog?
31. What is the difference between rand and randc?
32. How do you define soft constraints in SystemVerilog?
33. What is the difference between solve before and constraint_mode()?
34. How can you randomize an array with unique values?
35. How do you debug constraint failures in SystemVerilog?
36. What are the different phases in UVM?
37. What is a UVM factory, and why is it needed?
38. How does uvm_config_db work?
39. What is the purpose of uvm_sequence and uvm_sequence_item?
40. Explain the difference between a driver and a monitor in UVM.
41. What are the key differences between Verilog and SystemVerilog?
42. Explain logic vs. reg vs. wire.
43. What are always_ff, always_comb, and always_latch?
44. How do initial and always blocks differ?
45. What are packed and unpacked arrays in SystemVerilog?
46. Write a 4-bit adder using typedef struct packed.
47. Create a 3-state FSM using typedef enum logic.
48. Design a clock divider by N using parameter.
49. Create a 2:1 mux using always_comb.
50. Write a D flip-flop with synchronous reset using always_ff.
51. Create a packed struct for a packet header.
52. Define an enum for FSM states and use it in logic.
53. Use typedef to create a new type for 16-bit address.
54. Write a state machine using typedef enum with unique case.
55. Declare an array of packed structs.
56. Create an interface for a simple AXI-lite bus.
57. Use modport to separate read/write signals in interface.
58. Model a bus arbiter using typedef enum logic.

59. Code a priority encoder using case with unique.
60. Implement a watchdog timer using packed struct config.
61. Parameterize a register bank using typedef and generate.
62. Write a 2-flop CDC synchronizer using always_ff.
63. Write reset synchronizer with assertion for metastability.
64. Use localparam and generate if to conditionally instantiate logic.
65. Use assert property to check FSM output based on state.
66. Cover all outputs in FSM using covergroup.
67. Write a sequence to detect glitch on a control line.
68. Assert handshake doesn't occur more than once in 2 cycles.
69. Use disable iff to control assertion scope.
70. Design a UART receiver using interface.
71. Design an SPI controller with state machine.
72. Implement a basic APB slave using struct/interface.
73. Write FIFO with packed struct data format.
74. Model a register file using typedef for data/address.
75. Use package for constants and shared typedefs.
76. Implement a pulse synchronizer between domains.
77. Add assertion for synchronizer correctness.
78. Code handshake logic using struct and interface.
79. Write a parameterized N-bit counter.
80. Generate 8 DFFs using generate block.
81. Create a register bank of structs using parameterized size.
82. Instantiate an interface in a module and drive it.
83. Pass interface with modport into multiple modules.
84. Parameterize an interface for data width and address width.
85. Write an assertion to check setup of valid before ready.
86. Use a sequence to assert a 2-cycle delay between req and ack.
87. Assert no two masters access bus at same time.
88. Assert FSM never goes into an illegal state.
89. Cover a successful transaction using cover property.

90. Create a clocking block for testbench driving.
91. Write RTL code using clocking block for inputs and outputs.
92. Use input #1step delay in clocking block (for understanding).
93. Use @clk with clocking block.
94. Write a dual-edge triggered flip-flop using always_ff.
95. Create a 3-level nested struct for packet, header, and payload.
96. Use for-generate to instantiate FSMs per channel.

Topic 1: Randomization and Constraints

Q1. How do you generate a random variable between 10 and 50, restricted to even values only?

Answer: You can use SystemVerilog constraints inside a class with the ‘rand‘ keyword. Here is the solution:

```
class even_rand;
  rand bit [7:0] val;

  constraint even_c {
    val inside {[10:50]};
    val % 2 == 0;
  }
endclass

module tb;
  even_rand obj = new();
  initial begin
    repeat(5) begin
      void'(obj.randomize());
      $display("Even Value = %0d", obj.val);
    end
  end
endmodule
```

Explanation:

- ‘rand‘ declares the variable as randomizable.
- The ‘inside‘ operator constrains the range.
- ‘

Q2. How can you generate a unique array of 5 elements between 1 and 20?

Answer:

```
class unique_array;
  rand bit [4:0] arr[5];

  constraint c_range { foreach (arr[i]) arr[i] inside {[1:20]}; }
  constraint c_unique { unique {arr}; }
endclass

module tb;
  unique_array u = new();
  initial begin
    if (u.randomize())
      $display("Unique Array = %p", u.arr);
  end
endmodule
```

Explanation:

- 'unique' keyword ensures all elements are different.
- 'foreach' applies the range constraint to each element.

Q3. How to constrain a variable to only odd numbers from 5 to 15 but exclude consecutive odd values?

Answer:

```
class my_rand;
  rand bit [3:0] a;

  constraint c1 {
    a inside {[5:15]};
    a % 2 == 1;
    !(a == 7 || a == 9 || a == 11 || a == 13); // no consecutive odds
  }
endclass

module tb;
  my_rand obj = new();
  initial begin
```

```
    repeat(10) begin
      void'(obj.randomize());
      $display("a = %0d", obj.a);
    end
  end
endmodule
```

Explanation: The constraint excludes specific consecutive values to comply with the rule. This is a common interview challenge to test your understanding of exclusion logic in constraints.

Q4. Constrain two variables such that one is twice the other and their sum is less than 50.

Answer:

```
class rel_constraint;
  rand bit [7:0] a, b;

  constraint twice {
    a == 2 * b;
    a + b < 50;
  }
endclass

module tb;
  rel_constraint obj = new();
  initial begin
    repeat (5) begin
      void'(obj.randomize());
      $display("a=%0d, b=%0d, sum=%0d", obj.a, obj.b, obj.a + obj.b);
    end
  end
endmodule
```

Explanation: This type of relation constraint is crucial in protocol-level modeling when fields are mathematically dependent.

Q5. How do you write a constraint for sorting a 4-element array in ascending order?

Answer:

```
class sort_array;
  rand bit [7:0] arr[4];

  constraint c_sorted {
    arr[0] <= arr[1];
    arr[1] <= arr[2];
    arr[2] <= arr[3];
  }
endclass

module tb;
  sort_array s = new();
  initial begin
    if (s.randomize())
      $display("Sorted Array = %p", s.arr);
  end
endmodule
```

Explanation: Constraints are applied pairwise to enforce the sorting order. This is often used in test scenarios to ensure stimulus validity.

Q6. Generate a queue of size 5 containing even numbers only.

Answer:

```
class even_q;
  rand bit [7:0] q[$];

  constraint c1 {
    q.size() == 5;
    foreach (q[i]) q[i] % 2 == 0;
  }
endclass
```

```
module tb;
  even_q obj = new();
  initial begin
    if (obj.randomize())
      $display("Even Queue: %p", obj.q);
  end
endmodule
```

Explanation: Queues are dynamic arrays. The ‘foreach‘ loop ensures that each element satisfies the modulus constraint.

Q7. Randomize a 2D array in SystemVerilog such that each row is sorted.

Answer:

```
class array2D;
  rand bit [7:0] arr[3][4];

  constraint row_sorted {
    foreach (arr[i]) {
      foreach (arr[i,j]) if (j < 3)
        arr[i][j] <= arr[i][j+1];
    }
  }
endclass

module tb;
  array2D obj = new();
  initial begin
    if (obj.randomize())
      $display("2D Array = %p", obj.arr);
  end
endmodule
```

Explanation: Nested ‘foreach‘ constraints can be used to control the properties of multidimensional arrays. This ensures logical or protocol-based ordering.

Q8. Write a constraint such that the sum of array elements equals 100.

Answer:

```
class array_sum;
  rand bit [7:0] arr[5];

  constraint sum_c {
    arr.sum() == 100;
  }
endclass

module tb;
  array_sum obj = new();
  initial begin
    repeat (5) begin
      void'(obj.randomize());
      $display("Array = %p, Sum = %0d", obj.arr, obj.arr.sum());
    end
  end
endmodule
```

Explanation: ‘.sum()‘ is a built-in method used to calculate the total value of all elements in an array. This is especially useful for checksum generation and protocol timing scenarios.

Q9. Write a constraint such that each queue element is double the previous one.

Answer:

```
class exp_queue;
  rand bit [7:0] q[5];

  constraint exp {
    q[0] == 1;
    foreach(q[i]) if (i > 0) q[i] == 2 * q[i-1];
  }
```

```
endclass

module tb;
  exp_queue obj = new();
  initial begin
    if (obj.randomize())
      $display("Exponential Queue: %p", obj.q);
  end
endmodule
```

Explanation: This models geometric progressions useful in timing burst patterns or protocol test sequences.

Q10. Generate a queue from 0 to 9 with all unique values.

Answer:

```
class unique_q;
  rand bit [3:0] q[$];

  constraint c1 {
    q.size() == 10;
    foreach (q[i], q[j]) i != j -> q[i] != q[j];
    foreach (q[i]) q[i] inside {[0:9]};
  }
endclass

module tb;
  unique_q obj = new();
  initial begin
    if (obj.randomize())
      $display("Unique Queue: %p", obj.q);
  end
endmodule
```

Explanation: Constraints are used to ensure range and uniqueness, commonly used in generating randomized IDs or token sets.

Q11. Write a constraint to ensure sum of two variables is always even.

Answer:

```
class sum_even;
  rand bit [3:0] a, b;

  constraint even_sum { (a + b) % 2 == 0; }
endclass

module tb;
  sum_even obj = new();
  initial begin
    repeat (5) begin
      void'(obj.randomize());
      $display("a=%0d, b=%0d, sum=%0d", obj.a, obj.b, obj.a + obj.b);
    end
  end
endmodule
```

Explanation: Simple mathematical relations are a staple of DV testbench modeling. You'll often see parity or checksum conditions represented like this.

Q12. Create a constraint where one variable is always less than another, both in a given range.

Answer:

```
class less_than;
  rand bit [7:0] a, b;

  constraint limits { a inside {[10:50]}; b inside {[20:60]}; }
  constraint order { a < b; }
endclass

module tb;
  less_than obj = new();
  initial begin
```

```
    repeat (5) begin
      void'(obj.randomize());
      $display("a = %0d, b = %0d", obj.a, obj.b);
    end
  end
endmodule
```

Explanation: This is commonly used in timer delay settings or validation ranges where a dependent relationship exists.

Q13. How do you create a simple covergroup that captures a 4-bit data value into two bins: 0–7 (low) and 8–15 (high)?

Answer:

```
class transaction;
  rand bit [3:0] data;

  covergroup cg;
    option.per_instance = 1;
    DATA: coverpoint data {
      bins low = {[0:7]};
      bins high = {[8:15]};
    }
  endgroup

  function new();
    cg = new();
  endfunction

  function void sample_data();
    cg.sample();
  endfunction
endclass

module tb;
  transaction t = new();
  initial begin
```

```
    repeat(10) begin
      void'(t.randomize());
      t.sample_data();
      $display("Data = %0d", t.data);
    end
  end
endmodule
```

Explanation: Functional coverage uses covergroups and coverpoints to measure how thoroughly a stimulus space is exercised.

Q14. Create a covergroup to track transitions from IDLE to ACTIVE to DONE.

Answer:

```
typedef enum {IDLE, ACTIVE, DONE} state_t;
state_t state;

covergroup cg @(posedge clk);
  coverpoint state {
    bins full_path = (IDLE => ACTIVE => DONE);
  }
endgroup
```

Explanation: Transition bins are extremely useful in verifying protocol sequences and FSM transitions.

Q15. How do you sample a covergroup inside a class for protocol coverage?

Answer:

```
class bus_tx;
  rand bit [3:0] burst_len;

  covergroup burst_cg;
    coverpoint burst_len {
      bins single = {1};
```

```
      bins short  = {[2:4]};
      bins long   = {[5:15]};
    }
  endgroup

  function new();
    burst_cg = new();
  endfunction
endclass

module tb;
  bus_tx tx = new();
  initial begin
    repeat (10) begin
      void'(tx.randomize());
      tx.burst_cg.sample();
      $display("Burst Length = %0d", tx.burst_len);
    end
  end
endmodule
```

Explanation: This models burst length patterns, commonly required in protocol-level coverage models such as AXI or AHB.

Q16. How do you model cross coverage between two coverpoints in SystemVerilog?

Answer:

```
class cross_cov;
  rand bit [3:0] addr;
  rand bit [1:0] mode;

  covergroup cg;
    ADDR: coverpoint addr {
      bins low  = {[0:7]};
      bins high = {[8:15]};
    }
```

```
    MODE: coverpoint mode {
      bins rw = {0, 1};
      bins burst = {2, 3};
    }
    ADDR_MODE: cross ADDR, MODE;
  endgroup

  function new();
    cg = new();
  endfunction
endclass
```

Explanation: Cross coverage helps ensure that all combinations of two signals are exercised, which is vital in protocol verification.

Q17. What is 'option.per$_{i}$nstance'andwhyisitimportant?

Answer:

'option.per_instance = 1;' enables each instance of a class to have its own copy of the coverage data. Without it, all instances share a single covergroup result.

Use-case Example:

```
covergroup cg;
  option.per_instance = 1;
  coverpoint opcode;
endgroup
```

When to Use:

- In class-based models
- When collecting coverage from multiple environments or testcases

Q18. How do you ignore specific bins in a coverpoint?

Answer:

Use the 'ignore_bins' construct.

```
covergroup cg;
  coverpoint opcode {
    bins valid = {[0:15]};
```

```
    ignore_bins reserved = {6, 7};
  }
endgroup
```

Explanation: Ignored bins are excluded from coverage goals and are often used to filter out illegal or unused values.

Q19. How do you create automatic sampling on an event in a covergroup?

Answer:

```
covergroup auto_cg @(posedge clk);
  coverpoint state;
endgroup
```

Explanation: This allows coverage to be sampled automatically on clock edges or signal transitions, removing the need to call ‘.sample()‘ manually.

Q20. Write a constraint to randomize values divisible by 3 and in range 30 to 90.

Answer:

```
class div_by_3;
  rand bit [7:0] val;

  constraint c1 {
    val inside {[30:90]};
    val % 3 == 0;
  }
endclass

module tb;
  div_by_3 obj = new();
  initial begin
    repeat (10) begin
      void'(obj.randomize());
      $display("Value divisible by 3: %0d", obj.val);
```

```
    end
  end
endmodule
```

Explanation: The modulus operator is used to check divisibility. Combined with the ‘inside‘ operator, this restricts values to a specific range and multiple.

Q21. Generate a 3-element array such that the sum is even and elements are between 1 and 10.

Answer:

```
class sum_even_array;
  rand bit [3:0] arr[3];

  constraint range_c { foreach(arr[i]) arr[i] inside {[1:10]}; }
  constraint even_sum_c { arr.sum() % 2 == 0; }
endclass

module tb;
  sum_even_array obj = new();
  initial begin
    repeat (5) begin
      void'(obj.randomize());
      $display("Array = %p, Sum = %0d", obj.arr, obj.arr.sum());
    end
  end
endmodule
```

Explanation: The ‘.sum()‘ method calculates the array sum, and ‘

Q22. Constrain a variable to avoid a set of specific values.

Answer:

```
class avoid_vals;
  rand bit [7:0] val;

  constraint c1 {
```

```
    !(val inside {13, 27, 55});
    val inside {[10:60]};
  }
endclass

module tb;
  avoid_vals obj = new();
  initial begin
    repeat (5) begin
      void'(obj.randomize());
      $display("Value = %0d", obj.val);
    end
  end
endmodule
```

Explanation: Use '!inside' to exclude specific illegal values. Common in testbench scenarios where illegal or reserved values must be skipped.

Q31. What are the types of coverage in SystemVerilog?

Answer: SystemVerilog provides three main types of coverage:

- **Code Coverage:** Measures which parts of the code were executed (e.g., line, toggle, FSM).
- **Functional Coverage:** Measures whether all required functionality has been tested using coverage points.
- **Assertions Coverage:** Tracks whether assertions were triggered and whether pass/fail scenarios were covered.

Q32. What is functional coverage and why is it important?

Answer: Functional coverage is a user-defined metric that checks whether specific functional scenarios have been exercised during simulation. It is important because:

- It validates whether the design is tested thoroughly.
- It helps identify missing corner cases.
- It is used in coverage-driven verification.

Q33. What are covergroups in SystemVerilog?

Answer: A `covergroup` is a construct used to define functional coverage points. It includes:

- **coverpoints:** Variables or expressions to be monitored.
- **cross coverage:** Combination of two or more coverpoints.

Example:

```
covergroup cg;
  coverpoint op;
  coverpoint a;
  cross op, a;
endgroup
```

Q34. Explain the difference between coverpoint and cross coverage.

Answer:

- **Coverpoint:** Tracks the value distribution of a single variable.
- **Cross Coverage:** Measures combinations of values from two or more coverpoints to verify interaction between variables.

Q35. What is the significance of `bins` in coverage?

Answer: Bins group values into ranges or specific values for coverage tracking.

- **Implicit bins:** Created automatically.
- **Explicit bins:** User-defined for precise control.

Example:

```
coverpoint val {
  bins low = {[0:3]};
  bins mid = {[4:7]};
  bins high = {[8:15]};
}
```

Q36. What are coverage options in SystemVerilog?

Answer: Coverage options control how coverage data is collected or reported. Examples include:

- `option.per_instance` – Collects coverage per object instance.
- `option.weight` – Assigns weight to influence total coverage score.
- `option.comment` – Provides documentation.
- `option.name` – Customizes instance name in reports.

Q37. How to disable a coverpoint or covergroup dynamically?

Answer: You can disable a coverpoint or covergroup by:

- Setting `cg.option.weight = 0;` to ignore it in coverage.
- Not calling the `sample()` method when coverage is not required.

Q38. What is the use of `sample()` method in covergroups?

Answer: The `sample()` method captures current values of the coverpoints. It must be called explicitly if the covergroup is not declared with an `@` event. **Example:**

```
cg_inst.sample();
```

Q39. How do you use `illegal_bins` and `ignore_bins`?

Answer:

- `illegal_bins` – Flags undesired or invalid values.
- `ignore_bins` – Skips values in coverage calculation.

Example:

```
coverpoint mode {
  bins valid = {[0:3]};
  ignore_bins skip = {[4:5]};
  illegal_bins err = {[6:7]};
}
```

Q40. How can cross coverage be customized?

Answer: Cross coverage can be customized using:

- `bins` – To define valid combinations.
- `ignore_bins` or `illegal_bins` – To skip or flag combinations.
- `intersect` and `with` – To filter combinations with expressions.

Example:

```
cross a, b {
  ignore_bins ignore_ab = binsof(a) intersect {1} && binsof(b) intersect {3};
}
```

Q41. What is transition coverage in SystemVerilog?

Answer: Transition coverage tracks sequences of values a variable goes through. It helps ensure that legal state transitions occur. **Syntax Example:**

```
coverpoint state {
  bins s_trans[] = (S0 => S1 => S2);
}
```

Q42. How do you use wildcard bins in transition coverage?

Answer: Wildcards allow flexibility in defining transitions without exact values. **Example:**

```
bins wild[] = (1 => [*] => 3);
```

This matches a transition from 1 to 3 with any number of intermediate states.

Q43. What is the difference between `bins` and `bins[]` in transitions?

Answer:

- `bins name = (a => b);` defines a single transition.
- `bins name[] = (a => b => c);` creates a series of bins to capture each possible transition in the chain.

Q44. How can you collect coverage conditionally?

Answer: You can use `iff` clauses to conditionally enable coverage sampling. **Example:**

```
coverpoint addr iff (enable == 1);
```

This means `addr` will be sampled only if `enable` is 1.

Q45. How do you use coverage callbacks for advanced customization?

Answer: SystemVerilog does not directly support callbacks, but you can simulate similar behavior using class-based infrastructure and override `sample()` with custom logic before calling the actual sample.

Q46. What is `covergroup with` construct and how is it used?

Answer: `with` allows filtering or constraints on bins and cross coverage. **Example:**

```
cross a, b {
  bins cross_ab = binsof(a) intersect {1,2} &&
                  binsof(b) with (item < 5);
}
```

Q47. How do you merge coverage from multiple simulations?

Answer: Most simulators allow merging of multiple coverage databases using commands like:

- `vcover merge` (ModelSim/Questa)
- `urg -merge` (VCS)

This helps accumulate coverage over multiple test runs.

Q48. How to exclude certain bins or values from coverage reports?

Answer: Use `ignore_bins` to exclude values from affecting coverage percentage. **Example:**

```
ignore_bins idle_ignore = {[0:1]};
```

Q49. What are coverage directives and how are they used?

Answer: Directives are tool-specific pragmas used to control code coverage metrics. **Examples:**

- `// coverage off` – Turns off coverage for a block.
- `// coverage on` – Resumes coverage.

Q50. Can covergroups be parameterized or templated?

Answer: No, SystemVerilog does not support parameterized covergroups directly. However, you can achieve similar behavior by using arguments in the constructor to initialize different configurations. **Example:**

```
class cov;
  covergroup cg(int limit);
    coverpoint val {
      bins range[] = {[0:limit]};
    }
  endgroup
endclass
```

Q51. What is coverage-driven verification (CDV)?

Answer: CDV is a methodology where stimulus generation and verification effort are guided by coverage metrics. The aim is to maximize coverage efficiently, reducing redundant tests and improving quality.

Q52. How is functional coverage integrated with UVM?

Answer: In UVM, functional coverage is typically integrated using covergroups defined in sequence items, monitors, or scoreboards. The `uvm_subscriber` and `analysis_port` mechanism is commonly used to trigger coverage sampling.

Q53. What is the purpose of using `uvm_coverage` classes in UVM?

Answer: These classes help organize and encapsulate covergroups. They improve modularity and promote reuse by associating covergroups with the UVM testbench architecture.

Q54. How do you sample covergroups in UVM monitor or subscriber?

Answer: You call the `sample()` method after extracting or observing data. **Example:**

```
function void write(my_transaction tr);
  cov_inst.cp_inst.sample();
endfunction
```

Q55. Can coverpoints be triggered manually or automatically?

Answer: Coverpoints are always triggered manually using the `sample()` method. Unlike code coverage, functional coverage does not trigger automatically.

Q56. What are automatic bins and when are they useful?

Answer: Automatic bins are generated implicitly when a coverpoint range is specified without user-defined bins. They're useful for quickly covering wide ranges during early verification. **Example:**

```
coverpoint addr; // automatic bins based on type width
```

Q57. How do you measure cross coverage between signals from different interfaces?

Answer: You can create a central covergroup that includes coverpoints from both interfaces or pass values from both interfaces to a transaction class and cover them there.

Q58. What is the difference between `ignore_bins` and `illegal_bins`?

Answer:

- `ignore_bins` excludes values from coverage without raising any errors.
- `illegal_bins` raises a runtime error if those values are encountered, and they count as test failures.

Q59. How can you reduce simulation time while still collecting meaningful coverage?

Answer:

- Use targeted constraints to hit specific coverage goals.
- Use `ignore_bins` to skip irrelevant cases.
- Prioritize corner cases using weighted randomization.

Q60. How do you interpret functional coverage reports?

Answer: Coverage reports list:

- Covergroups and coverpoints with hit counts.
- Percentage covered.
- Missing bins and transitions.

Analyzing this helps guide regression planning and identify unverified behavior.

Q61. What is an assertion in SystemVerilog and why is it used?

Answer: An assertion in SystemVerilog is a construct used to validate the behavior of a design by checking for expected conditions at runtime. Assertions help catch design bugs early by ensuring that certain conditions hold true during simulation. They improve design reliability and simplify debugging by localizing violations of intended behavior.

Q62. What are the two types of assertions in SystemVerilog? Explain each.

Answer: SystemVerilog supports two main types of assertions:

- **Immediate Assertions**: These are evaluated immediately when the statement is executed. They are typically used in procedural code like initial or always blocks.
- **Concurrent Assertions**: These evaluate properties over time and are used to monitor behavior across clock cycles. They are written using temporal expressions.

Q63. What are the primary methods used with concurrent assertions?

Answer: Concurrent assertions use the following keywords/methods:

- **assert property**: Checks if a property holds.
- **assume property**: Used in formal verification; assumes the property holds.
- **cover property**: Used to ensure that a property occurs during simulation for functional coverage.

Q64. What is the significance of the "disable iff" construct in assertions?

Answer: The `disable iff` clause is used in concurrent assertions to disable or ignore the evaluation of the assertion when a certain condition is true. It's typically used to avoid false failures during reset or when specific invalid conditions are present.

```
assert property (@(posedge clk) disable iff (reset) (req |=> ack));
```

In this example, the assertion is not evaluated when `reset` is high.

Q65. What are the benefits of using assertions in a verification environment?

Answer: Assertions provide several benefits:

- Early detection of design bugs.
- Better documentation of design intent.
- Reduced debugging time.
- Enhanced readability and maintainability.
- Enables formal verification and simulation coverage tracking.

Q66. Write a concurrent assertion to check that whenever 'req' goes high, 'ack' should go high in the next cycle.

Answer:

```
property req_ack;
  @(posedge clk) req |=> ack;
endproperty

assert property (req_ack);
```

This assertion checks that if 'req' is high on one clock cycle, 'ack' must be high on the next clock cycle.

Q67. How do you assert that a signal 'data_valid' remains high for 3 consecutive clock cycles once it becomes high?

Answer:

```
property data_valid_stable;
  @(posedge clk) data_valid |=> data_valid [*2];
endproperty

assert property (data_valid_stable);
```

The repetition operator ‘[*2]‘ checks that ‘data_valid‘ remains high for 2 more cycles after the initial high (total of 3).

Q68. Write an assertion to ensure that a signal ‘write‘ is not active during reset.

Answer:

```
property no_write_on_reset;
  @(posedge clk) disable iff (reset) !write;
endproperty

assert property (no_write_on_reset);
```

The assertion ensures that ‘write‘ is not asserted when reset is active.

Q69. How can you use the ‘cover property‘ to check that a transaction of ‘start ¿ done‘ happens at least once during simulation?

Answer:

```
property start_done_transaction;
  @(posedge clk) start |=> done;
endproperty

cover property (start_done_transaction);
```

The ‘cover property‘ is used for functional coverage to ensure the ‘start -¿ done‘ scenario occurs during simulation.

Q70. Write an assertion that checks a burst transfer of 4 cycles starting from ‘burst_start‘ with ‘data_valid‘ high in all cycles.

Answer:

```
property burst_transfer;
  @(posedge clk) burst_start |=> data_valid [*4];
endproperty

assert property (burst_transfer);
```

This checks that after ‘burst_start‘, ‘data_valid‘ remains high for 4 consecutive cycles, indicating a proper burst transfer.

Q76. Explain how the ‘disable iff‘ clause works in assertions and provide an example.

Answer:

The ‘disable iff‘ clause disables the assertion when its condition is true, effectively masking the assertion temporarily.

Example:

```
property p;
  @(posedge clk) disable iff (reset) (a |=> b);
endproperty

assert property (p);
```

Here, the assertion is disabled (ignored) whenever ‘reset‘ is high.

Q77. What is the difference between immediate and concurrent assertions? Provide examples.

Answer:

- **Immediate assertions** execute instantly at the point in procedural code. - **Concurrent assertions** run continuously alongside simulation time, checking temporal properties.

Examples:

Immediate assertion:

```
always @(posedge clk) begin
  assert (a == b) else $error("Mismatch");
end
```

Concurrent assertion:

```
property p;
  @(posedge clk) a |=> b;
endproperty

assert property (p);
```

Q78. How can you parameterize assertions in SystemVerilog? Provide a simple example.

Answer:

Assertions can be parameterized using ‘parameter‘ to make them reusable with different conditions.

Example:

```
parameter int WIDTH = 8;

property p #(parameter int W = WIDTH);
  @(posedge clk) (data == W) |=> (valid == 1);
endproperty

assert property (p#(16));
```

Q79. Describe the use of ‘cover property‘ in assertions. How is it different from ‘assert property‘?

Answer:

- ‘assert property‘ checks that a property holds true; simulation reports failure on violation. - ‘cover property‘ tracks if a property ever becomes true during simulation, useful for coverage analysis.

Example:

```
cover property (@(posedge clk) a ##1 b);
```

This records coverage when ‘a‘ is followed by ‘b‘ after 1 cycle.

Q80. What is a temporal implication (‘—=¿‘) in SVA, and how does it differ from logical implication (‘-¿‘)?

Answer:

- Temporal implication (‘—=¿‘) means ”if antecedent sequence happens now, then consequent sequence must happen starting next clock cycle.” - Logical implication (‘-¿‘) is a combinational operator without timing semantics.

Example:

```
property p;
  @(posedge clk) (a ##1 b) |=> (c ##1 d);
endproperty
```

Here, if ‘a 1 b‘ occurs, then ‘c 1 d‘ must follow in the subsequent cycles.

Logical implication example:

```
assert (a -> b); // immediate logic check
```

No timing is involved in logical implication.

Q81: What is Object-Oriented Programming (OOP) in SystemVerilog?

Answer:

OOP in SystemVerilog is a programming paradigm that uses objects and classes to model

real-world entities and concepts. It supports encapsulation, inheritance, and polymorphism to improve code reuse, modularity, and maintainability in verification environments.

Q82: What are the main features of OOP in SystemVerilog?

Answer:

The main features are:

- Encapsulation: Bundling data and methods in classes.
- Inheritance: Deriving new classes from existing ones to reuse and extend functionality.
- Polymorphism: Ability to call methods of derived classes through base class handles (dynamic dispatch).
- Abstraction: Hiding implementation details behind interfaces and classes.

Q83: What is a class in SystemVerilog? How do you define it?

Answer:

A class is a blueprint for creating objects containing data (variables) and behavior (methods). It is defined using the `class` keyword.

Example:

```
class Packet;
  int data;
  function void display();
    $display("Data = %0d", data);
  endfunction
endclass
```

Q84: How do you create an object from a class in SystemVerilog?

Answer:

You create an object using the `new` operator on a class handle.

Example:

```
Packet pkt;
pkt = new();
pkt.data = 10;
pkt.display();
```

Q85: What is a constructor in SystemVerilog classes?

Answer:

A constructor is a special function named `new` that initializes the object at creation. It can take arguments to set initial values.

Example:

```
class Packet;
  int data;
  function new(int d);
    data = d;
  endfunction
endclass
```

Q86: What is inheritance in SystemVerilog? How is it implemented?

Answer:

Inheritance allows a class (derived class) to acquire properties and methods of another class (base class). Implemented using the `extends` keyword.

Example:

```
class BaseClass;
  function void show();
    $display("Base class");
  endfunction
endclass

class DerivedClass extends BaseClass;
  function void show();
    $display("Derived class");
  endfunction
endclass
```

Q87: What is polymorphism in SystemVerilog?

Answer:

Polymorphism allows methods to behave differently depending on the object's actual derived type, even if accessed through a base class handle. Achieved using virtual methods and dynamic dispatch.

Q88: How do you declare and use virtual methods?

Answer:

Declare a method with the `virtual` keyword in the base class. Derived classes override it. When called via a base class handle, the derived version executes at runtime.
Example:

```
class BaseClass;
  virtual function void display();
    $display("Base");
  endfunction
endclass

class DerivedClass extends BaseClass;
  function void display();
    $display("Derived");
  endfunction
endclass

BaseClass obj;
obj = new DerivedClass();
obj.display(); // Prints "Derived"
```

Q89: What are class handles and how do they differ from variables?

Answer:

Class handles are pointers/references to objects. Variables hold actual data. Class handles allow dynamic memory allocation, sharing, and polymorphism.

Q90: How is memory managed in SystemVerilog OOP?

Answer:

Objects are created dynamically using `new`. The simulator automatically garbage collects unreferenced objects, but explicit `delete` can be used to free objects sooner.

1 Data Types and Casting

1. Explain the differences between ‘logic‘, ‘reg‘, ‘wire‘, ‘bit‘, and ‘byte‘ in SystemVerilog.
2. How does SystemVerilog handle signed versus unsigned arithmetic by default?
3. Describe the four states of a 4-state data type and how they differ from a 2-state type.
4. What are implicit casts and when might they lead to unexpected behavior?
5. Compare and contrast ‘$cast‘ and ‘cast<type>()‘. Provide use-case examples.
6. How would you define a packed ‘struct‘ and how does alignment affect it?
7. Explain the behavior of ‘union‘ in SystemVerilog, especially regarding overlapping memory.
8. What is the difference between static and automatic variables inside functions and tasks?
9. How does SystemVerilog manage overflow and underflow in arithmetic operations?
10. When and why would you use ‘typedef‘ for parameterized types?

2 Processes and Concurrency

1. Describe the differences between ‘always_ff‘, ‘always_comb‘, and ‘always_latch‘.
2. Explain blocking (‘=‘) versus non-blocking (‘<=‘) assignments and typical use-cases.
3. How do delta cycles and event scheduling work in SystemVerilog simulation?
4. What are race conditions, and how can you avoid them?
5. How do you generate an automatic sensitivity list for combinational logic?
6. Explain the semantics of ‘fork...join‘, ‘fork...join_any‘, and ‘fork...join_none‘.
7. How can you safely terminate or disable a running process?
8. Discuss the use of explicit ‘@‘ event control versus sensitivity lists.
9. What mechanisms exist for prioritizing processes or events?
10. How would you use an unresolved ‘wire‘ to model multiple drivers?

3 Interfaces and Modports

1. What is a SystemVerilog ‘interface‘ and what problems does it solve?
2. How do you connect an ‘interface‘ instance to a module port?
3. Explain the purpose of ‘modport‘ and how it restricts signal access.
4. Describe ‘clocking‘ blocks within an interface and their benefits.
5. When and how would you use a ‘virtual interface‘?
6. How does the ‘bind‘ statement work with interfaces?
7. What is a parameterized interface, and how do you declare one?
8. How do you package and reuse an interface across multiple modules?
9. Compare using an ‘interface‘ versus individual port lists.
10. How can you synchronize reset and clock using constructs in an interface?

4 Constraints and Randomization

1. Explain the difference between ‘rand‘ and ‘randc‘ variables.
2. How do you constrain a random variable to a specific range?
3. What happens when two constraints conflict? How do you debug it?
4. Describe soft constraints and their typical use-cases.
5. Outline the solver phases in SystemVerilog randomization.
6. How do you randomize dynamic arrays and queues?
7. Explain conditional constraints with an example.
8. What is constraint coverage and why is it useful?
9. How can you seed the randomization process for reproducibility?
10. Describe using ‘solve...before‘ to order constraint evaluation.

5 Classes and Object-Oriented Programming

1. How do you declare a class and create an object instance in SV?
2. What are static class members and when would you use them?
3. Explain inheritance and polymorphism in SystemVerilog.
4. What is a virtual method and how is it declared?
5. How do you call a parent class constructor using ‘super‘?

6. Describe type parameterization in classes with an example.
7. When would you declare a method as ‘pure virtual‘?
8. How is ‘clone()‘ used to duplicate class objects?
9. Discuss garbage collection and object lifetime management.
10. How can you implement a factory pattern in SV classes?

6 Functional Coverage

1. Define functional coverage and its role in verification.
2. How do you declare a ‘covergroup‘ and ‘coverpoint‘?
3. Explain bins and wildcard bins in coverage.
4. How do you cross two ‘coverpoints‘?
5. What is coverage sampling and how is it performed?
6. How do you control coverage options like ‘option.weight‘?
7. Describe using coverage directives to exclude illegal states.
8. How can you report coverage holes programmatically?
9. Discuss strategies to achieve coverage closure.
10. How do you integrate coverage data into a testbench flow?

7 Sequences and Drivers

1. Explain how you would generate a sequence of transactions using tasks and ‘fork...join‘.
2. How do you randomize the order of transactions in a sequence?
3. Describe using a clocking block to synchronize sequence items.
4. How would you implement timeouts within a sequence to avoid hangs?
5. Explain the use of ‘mailbox‘ for communication between sequence and driver.
6. How can you parameterize a driver task for multiple test scenarios?
7. Discuss handling back-to-back transactions in a pipelined driver.
8. How do you monitor and check transaction timing in a driver?
9. What techniques can you use to pause and resume a driver task on an event?
10. How do you implement a variable delay between transactions?

8 Scoreboarding and Analysis

1. What is a scoreboard and why is it critical in verification?
2. How do you implement a scoreboard using associative arrays?
3. Describe comparing DUT output to a reference model.
4. How can you handle out-of-order transactions in a scoreboard?
5. Explain techniques for tolerant comparison (e.g., bitmasks).
6. How do you reset and reinitialize scoreboard state between tests?
7. Discuss integrating functional coverage within a scoreboard.
8. How can you generate a summary report of mismatches?
9. What strategies exist for concurrent scoreboarding across agents?
10. How do you debug and trace scoreboard failures effectively?

1 Advanced Data Types and Structures (Q1–Q25)

Q1. What is the difference between ‘struct‘, ‘union‘, and ‘class‘ in SystemVerilog? Provide examples.

Answer: SystemVerilog supports three primary composite data types: ‘struct‘, ‘union‘, and ‘class‘. Here’s a detailed comparison:

- **struct** groups variables under a single name; all variables exist simultaneously.
- **union** allows only one variable to occupy memory at any time (overlapping memory).
- **class** is a dynamic object with OOP features such as inheritance and polymorphism.

Example:

```
typedef struct {
   bit [3:0] id;
   bit [7:0] data;
} packet_t;

typedef union {
   bit [31:0] raw;
   struct {
     bit [7:0] header;
     bit [23:0] payload;
   } fields;
} u_data;

class transaction;
   rand bit [7:0] addr;
   rand bit [31:0] data;
endclass
```

Key Points:

- ‘struct‘ and ‘union‘ are used for static memory-efficient design.
- ‘class‘ allows OOP principles and dynamic behavior, making it ideal for testbench modeling.

Q2. Explain ‘typedef struct packed‘ and its practical use in design/verification.

Answer: The ‘typedef struct packed‘ keyword defines a packed structure, where all members are packed next to each other with no padding, allowing it to be treated like a single vector.

Syntax:

```
typedef struct packed {
   logic [3:0] opcode;
   logic [11:0] addr;
   logic [15:0] data;
} instruction_t;
```

Why Use It?

- Packed structs can be assigned to bit vectors directly.
- Useful for representing bus protocols and aligning with memory layouts.

Example Use:

```
instruction_t instr;
logic [31:0] raw_instr;

raw_instr = 32'hABCD1234;
instr = instruction_t'(raw_instr);
```

Q3. Differentiate between ‘static‘, ‘automatic‘, and ‘dynamic‘ arrays. Which is best suited in testbenches?

Answer:

- **Static arrays:** Size known at compile-time.
- **Automatic arrays:** Stack-allocated arrays created fresh each time a task is called.
- **Dynamic arrays:** Size can be changed during runtime using ‘new[]‘.

  Example:

```
bit [7:0] static_array[4];              // Static
automatic bit [7:0] auto_array[4];      // Automatic
bit [7:0] dyn_array[];                  // Dynamic

initial begin
  dyn_array = new[8]; // Allocates size at runtime
end
```

- **Best for Testbenches:** Dynamic arrays — provide flexibility and memory efficiency.

Q4. What is a ‘queue‘ in SystemVerilog and how is it different from a dynamic array?

Answer: A ‘queue‘ is a variable-size, ordered collection similar to a dynamic array but supports push/pop operations.

Key Differences:

- Queues grow and shrink with push_front(), push_back(), pop_front(), pop_back().
- Dynamic arrays only resize via ‘new[].

Syntax:

```
bit [7:0] q[$]; // Declaring a queue

initial begin
   q.push_back(10); // Add to end
   q.push_front(5); // Add to start
   q.pop_back();    // Remove last element
end
```

Use Case: Queues are ideal for modeling FIFOs or temporary transaction buffers.

Q5. What are associative arrays in SV? Where are they best used?

Answer: Associative arrays allow indexing using any scalar data type, not just integers. They're best when the index is sparse or unknown at compile time.

Syntax:

```
bit [7:0] aa[string]; // Associative array with string key

initial begin
   aa["read"] = 1;
   aa["write"] = 0;
end
```

Features:

- Dynamic memory
- Iterator support: foreach, first(), last(), next(), prev()

Used in: Scoreboards, coverage models, response tracking.

Q6. How are associative arrays iterated in SystemVerilog? Give an example.

Answer: SystemVerilog supports four iterator methods for associative arrays:

- first(var)
- last(var)
- next(var)
- prev(var)

Example:

```
bit [7:0] aa[string];
string key;

initial begin
    aa["a"] = 1;
    aa["b"] = 2;
    aa["c"] = 3;

    key = aa.first();
    while (key != "") begin
        $display("Key: %s, Value: %0d", key, aa[key]);
        key = aa.next(key);
    end
end
```

Q7. Explain the use of '$size', '$left', '$right', '$high', '$low' system tasks with respect to arrays.

Answer: These system tasks provide bounds and size-related information of arrays.

- $size(array) - Numberofelements$.left(array) - Left-most index.
- $right(array) - Right - mostindex$.low(array) - Lower bound.

- $high(array) - Upperbound$.
  Example:

```
bit [7:0] arr[5:1];

initial begin
  $display("Size: %0d", $size(arr));    // Output: 5
  $display("Left: %0d", $left(arr));    // Output: 5
  $display("Right: %0d", $right(arr));  // Output: 1
  $display("Low: %0d", $low(arr));      // Output: 1
  $display("High: %0d", $high(arr));    // Output: 5
end
```

Q8. How are queues initialized and resized during simulation? Can they shrink dynamically?

Answer: Yes. Queues are self-sizing and can be dynamically modified using 'push$_f$*ront*', '*push$_b$ack*', '*po*

Example:

```
bit [7:0] q[$];

initial begin
  q = {1, 2, 3};
  q.push_back(4);
  q.pop_front();
  q.pop_back();
end
```

Note: Use queues in scoreboards and transaction-level modeling for dynamic buffering.

Q9. Write a function to reverse a dynamic array in SystemVerilog.

Answer:

```
function void reverse_array(ref int dyn[]);
   int i, temp;
   for (i = 0; i < dyn.size()/2; i++) begin
      temp = dyn[i];
      dyn[i] = dyn[dyn.size()-1 - i];
      dyn[dyn.size()-1 - i] = temp;
   end
endfunction
```

Usage:

```
int a[] = {1,2,3,4,5};
reverse_array(a);
```

Q10. What are ‘bit-stream‘ casting and its advantages? Provide an example.

Answer: Bit-stream casting is used to reformat bits into another type without changing the underlying bit pattern.

Syntax:

```
type'(bit_vector)
```

Example:

```
typedef struct packed {
   bit [7:0] addr;
   bit [7:0] data;
} packet_t;

bit [15:0] raw = 16'hABCD;
packet_t pkt = packet_t'(raw);
```

Advantage:

- Easy and efficient reinterpretation of data.
- Used for modeling memory-mapped protocols.

Q11. How does a class handle memory in SystemVerilog? Explain shallow vs deep copy.

Answer: In SystemVerilog, ‘class‘ objects are dynamic and accessed via handles. Two types of copy behavior exist:

- **Shallow Copy:** Copies handle only. Both variables point to the same memory.
- **Deep Copy:** Clones object and creates a new instance with same data.

Example (Shallow Copy):

```
class A;
  int x;
endclass

A obj1 = new();
obj1.x = 10;

A obj2 = obj1; // Shallow copy
obj2.x = 20;   // obj1.x is also 20
```

Q12. How is new[] used in dynamic arrays and classes? Show with both.

Answer: new[] dynamically allocates memory for arrays or array of class objects.

Dynamic Array:

```
int dyn_array[];
dyn_array = new[10]; // Allocates 10 elements
```

Class Array:

```
class Item;
  int value;
endclass

Item items[];
initial begin
  items = new[5];
  foreach(items[i])
    items[i] = new(); // Allocate each object
end
```

Q13. Explain the use of foreach loop in iterating over arrays. Give all valid variants.

Answer: foreach is a SystemVerilog looping construct optimized for arrays (including multidimensional).

1D Example:

```
int arr[5];

foreach (arr[i])
   $display("Index: %0d, Value: %0d", i, arr[i]);
```

2D Example:

```
bit [7:0] arr[3][4];

foreach (arr[i,j])
   $display("arr[%0d][%0d]=%0d", i, j, arr[i][j]);
```

Q14. Can we pass associative arrays to functions? How are they passed and accessed?

Answer: Yes, associative arrays can be passed to functions/tasks by reference.

Syntax:

```
function void printAA(ref int aa[string]);
   string key;
   key = aa.first();
   while (key != "") begin
      $display("%s = %0d", key, aa[key]);
      key = aa.next(key);
   end
endfunction
```

Call:

```
int aa[string];
aa["apple"] = 1;
aa["banana"] = 2;
printAA(aa);
```

Q15. Differentiate between class and interface in SV with use cases.

Answer:

- **Class** is used in testbenches for transaction modeling, sequences, and OOP.
- **Interface** encapsulates signal-level connectivity, grouping logic signals.

Class:

```
class Packet;
  rand bit [7:0] addr;
endclass
```

Interface:

```
interface bus_if;
  logic clk, rst;
  logic [7:0] data;
endinterface
```

Usage: Class is behavioral, interface is structural.

Q16. Explain the difference between rand and randc in SystemVerilog. When is each used?

Answer:

- **rand**: Generates random values with repetition allowed.
- **randc**: Generates random values in a cyclic fashion (without repetition until all values are used).

Example:

```
class A;
  rand bit [1:0] x;     // can repeat
  randc bit [1:0] y;    // cycles through all values 0 to 3
endclass
```

Use Case: Use ‘randc‘ when you want exhaustive permutation, such as generating all priority levels, IDs, etc.

Q17. What is constraint overriding and how is it achieved in SystemVerilog?

Answer: Constraint overriding allows a child class to override constraints defined in a parent class.

Example:

```
class Base;
  rand bit [7:0] data;
  constraint c1 { data < 100; }
endclass

class Child extends Base;
  constraint c1 { data > 150; } // Overrides parent
endclass
```

Note: Constraint names must match exactly. Overridden constraints are evaluated instead of the parent's.

Q18. Can randomization fail in SystemVerilog? How do you detect and debug it?

Answer: Yes. Randomization can fail if constraints conflict or there are no legal values possible.

Detection:

```
if (!obj.randomize())
  $error("Randomization Failed");
```

Debugging Tips:

- Reduce constraints step-by-step.
- Print constraints and variables.

- Use constraint solver diagnostic tools (e.g., *display*, *coverage*).

Q19. What is a randcase and how is it different from case in SystemVerilog?

Answer: case selects based on expression values. **randcase** selects based on probabilistic weights.

Example:

```
randcase
  1: a = 1;
  3: a = 2; // Higher weight, more likely
endcase
```

Use Case: Modeling probabilistic behaviors such as random traffic generation, error injection, etc.

Q20. How does inheritance work in SystemVerilog classes? Include a code example.

Answer: SystemVerilog supports single inheritance using ‘extends‘. The derived class inherits all properties of the base class.

Example:

```
class Animal;
  function void speak();
    $display("Generic Animal");
  endfunction
endclass

class Dog extends Animal;
  function void speak();
    $display("Bark");
  endfunction
endclass
```

Usage:

```
Animal a = new Dog();
a.speak(); // Outputs: Bark
```

Note: Dynamic dispatch applies, enabling polymorphism.

Q21. What is the difference between ‘static‘ and ‘automatic‘ variables in SystemVerilog?

Answer:

- **Static:** Retains value across function/task calls.
- **Automatic:** Allocated on stack, initialized every call.

Example:

```
function void static_counter();
  static int x = 0;
  x++;
  $display("Static x = %0d", x);
endfunction

function void auto_counter();
  automatic int y = 0;
  y++;
  $display("Auto y = %0d", y);
endfunction
```

Q22. Explain virtual classes in SystemVerilog with an example.

Answer: A virtual class is an abstract class meant to be inherited and never instantiated directly.

Use Case: Common base for polymorphic behavior.

Example:

```
virtual class Packet;
  pure virtual function void send();
endclass

class EthernetPacket extends Packet;
  function void send(); $display("Ethernet sent"); endfunction
endclass
```

Instantiation:

```
Packet p = new EthernetPacket();
p.send(); // Output: Ethernet sent
```

Q23. Can class handles be assigned to null? How is null handle dereferencing handled?

Answer: Yes, class handles default to 'null'. Dereferencing a null handle causes a runtime fatal error.

Example:

```
class C;
  int x;
endclass

C obj = null;
$display(obj.x); // Fatal: null handle dereference
```

Prevention:

```
if (obj != null)
  $display(obj.x);
```

Q24. What is a copy constructor in SystemVerilog and how is it implemented?

Answer: A copy constructor duplicates one object's values into another object during instantiation.

Syntax:

```
class Packet;
  int data;
  function new(Packet p);
    this.data = p.data;
  endfunction
endclass
```

Usage:

```
Packet p1 = new();
p1.data = 100;
Packet p2 = new(p1); // Copy constructor
```

Tip: Manually copy nested members for deep copy.

Q25. How are constraints prioritized in SystemVerilog? What happens if multiple constraints affect a variable?

Answer: SystemVerilog doesn't prioritize constraints explicitly. All active constraints are processed by the solver to find a legal solution. If constraints conflict, randomization fails.

Example:

```
rand int x;
constraint c1 { x > 5; }
constraint c2 { x < 3; } // Conflicts with c1 → randomization fails
```

Note: Avoid overlapping or contradictory constraints.